Nmap Integration — Added Files, Testing & Operator Guide

Cerberus Pentest Framework · External Tool #2 (Nmap + Metasploit Coexistence) · Date: October 27, 2026 (2026-10-27)

commit 8816479 · 2026-10-27 · 9 files (+938 lines) · 65/65 framework & tool tests passing

This document records every file added and updated to integrate **Nmap (`https://github.com/nmap/nmap`)** into the Cerberus Pentest Framework alongside **Metasploit**, how to run automated and live tests in VS Code, and how to use all Nmap features (port discovery, service/version detection, CPE extraction, safe NSE scripts, and cross-tool state sharing).

1 Files Created & Modified (Commit `8816479` — 2026-10-27)

Zero lines of `metasploit_adapter.py` or `metasploit_scanner.py` were modified. Nmap plugs into the exact same `ToolAdapter` + `AttackModule` contract and is auto-discovered at startup:

File Path	Status	Lines	Purpose & Responsibilities
`backend/app/ai_engine/pentest/tools/nmap_adapter.py`	NEW	354	`NmapAdapter(ToolAdapter)` — invokes `nmap` via `asyncio.create_subprocess_exec` (never a shell), runs `-sT -Pn -T4 -sV --version-light -oX -` , parses `<nmaprun>` XML via stdlib `xml.etree.ElementTree` , blocks flag injection, and refuses intrusive NSE scripts unless RoE grants FULL impact.
`backend/app/ai_engine/pentest/modules/tools/nmap_scanner.py`	NEW	289	`NmapScannerModule(AttackModule)` (`name="nmap_scanner"` , category `tools` , MITRE `T1046/T1592`). Enforces opt-in check, scope-approved port filtering, RoE script gates (`dos` & `exploit/intrusive` refused), fail-soft on missing binary, and shares `nmap_open_ports` & `nmap_services` into `context.shared_state` .
`backend/tests/unit/test_pentest_tools.py`	MODIFIED	+269 (589)	Keeps all 15 Metasploit tests intact and adds 12 new tests (`FakeNmap` hermetic CLI simulator, XML parsing, NSE script gates, flag-injection guard, fail-soft, and joint `metasploit_scanner + nmap_scanner` execution in one plan).
`backend/app/core/config.py`	MODIFIED	+11	Adds `NMAP_ENABLED: bool = False` , `NMAP_PATH: str = "nmap"` , and `NMAP_TIMEOUT: int = 120` alongside `MSF_*` settings.
`backend/.env.example`	MODIFIED	+7	Documents `NMAP_ENABLED=false` , `NMAP_PATH=nmap` , and `NMAP_TIMEOUT=120` .
`backend/app/ai_engine/pentest/tools/__init__.py`	MODIFIED	+2	Exports `NmapAdapter` alongside `MetasploitAdapter` .
`backend/app/ai_engine/pentest/modules/tools/__init__.py`	MODIFIED	+2	Updates package docstring for multi-tool modules (Metasploit + Nmap).
`backend/app/ai_engine/pentest/__init__.py`	MODIFIED	+2	Exports `NmapAdapter` at the top-level `app.ai_engine.pentest` package.
`scripts/live_tests/pentest_framework_smoke.py`	MODIFIED	+15	Runs both `metasploit_scanner` and `nmap_scanner` during the live smoke test and verifies `tools registered: 2` and `modules registered: 8` .

2 How Nmap & Metasploit Coexist in the Framework Architecture

```
ExecutionScheduler.run(plan, context)
  │
  ├──► [1] ExecutionEngine.execute("nmap_scanner", context)
  │       ├── SafetyGuard.preflight()  ──► checks target in ScopeConfig.allowed_targets
  │       ├── NmapScannerModule        ──► filters ports via ScopeConfig.is_port_allowed() + checks NSE scripts
  │       ├── NmapAdapter              ──► runs: nmap -sT -Pn -T4 -p <ports> -sV --version-light -oX - <host>
  │       └── Shares into context:     ──► context.share_data("nmap_open_ports", [...]) & "nmap_services"
  │
  └──► [2] ExecutionEngine.execute("metasploit_scanner", context)
          ├── SafetyGuard.preflight()  ──► checks target in ScopeConfig.allowed_targets
          ├── MetasploitScannerModule  ──► enforces auxiliary/* gate (blocks exploit/* under PoC RoE)
          └── MetasploitAdapter        ──► runs module over XML-RPC (msfrpcd) & returns structured Finding
```

3 How to Install & Configure Nmap (MacBook & Windows)

1A. Install Nmap on MacBook (`zsh`):

```
brew install nmap
nmap --version
```

1B. Install Nmap on Windows (PowerShell):

```
winget install Insecure.Nmap
nmap --version
```

2. Enable Nmap (and/or Metasploit) in `backend/.env` :

```
# Nmap external tool settings (backend/.env)
NMAP_ENABLED=true
NMAP_PATH=nmap
NMAP_TIMEOUT=120

# Metasploit external tool settings (can be enabled at the same time)
MSF_ENABLED=true
MSF_RPC_HOST=127.0.0.1
MSF_RPC_PORT=55553
MSF_RPC_PASSWORD=YourStrongPassword
MSF_RPC_SSL=false
```

> **Fail-soft guarantee:** Even if `nmap` is not installed on the machine yet, all 27 unit tests and the hermetic multi-tool scripts below work out of the box using `FakeNmap` , and if `NMAP_ENABLED=true` without the binary installed, `nmap_scanner` safely returns an `INFO` finding (`Nmap unavailable`) without ever crashing the engagement.

4 Step-by-Step Verification Tests in VS Code Terminal

Test A — Run All 27 External Tool Unit Tests (15 Metasploit + 11 Nmap + 1 Joint Multi-Tool Test):

```
cd backend
source .venv/bin/activate   # (or .\.venv\Scripts\Activate.ps1 on Windows PowerShell)
pytest --noconftest tests/unit/test_pentest_tools.py -v
```

Expected output:

```
tests/unit/test_pentest_tools.py::test_tool_registry_discovers_metasploit PASSED
tests/unit/test_pentest_tools.py::test_tool_registry_discovers_nmap PASSED
tests/unit/test_pentest_tools.py::test_nmap_adapter_connects_and_reports_version PASSED
tests/unit/test_pentest_tools.py::test_nmap_adapter_executes_scan_and_parses_xml PASSED
tests/unit/test_pentest_tools.py::test_nmap_adapter_refuses_intrusive_scripts PASSED
tests/unit/test_pentest_tools.py::test_nmap_adapter_rejects_flag_injection_in_host PASSED
tests/unit/test_pentest_tools.py::test_nmap_module_runs_scan_via_engine PASSED
tests/unit/test_pentest_tools.py::test_nmap_module_refuses_intrusive_scripts PASSED
tests/unit/test_pentest_tools.py::test_nmap_module_refuses_destructive_dos_scripts PASSED
tests/unit/test_pentest_tools.py::test_metasploit_and_nmap_run_together_without_conflict PASSED
======================== 27 passed in ~7s ========================
```

Test B — Run the Full 65-Test Pentest Framework + Tools Suite & Smoke Script:

```
pytest --noconftest -k "not persist_findings" tests/unit/test_pentest_framework.py tests/unit/test_pentest_tools.py -q
python ../scripts/live_tests/pentest_framework_smoke.py
```

Expected output: `65 passed` , followed by the smoke test confirming `modules registered: 8` and `tools registered: 2` (`metasploit` , `nmap`).

5 Run Nmap + Metasploit Together in One Engagement Plan (Hermetic — No External App Needed)

Paste this into your activated VS Code terminal (`backend/`) to prove that `nmap_scanner` and `metasploit_scanner` execute sequentially in the same `ExecutionScheduler` plan, share discovered ports, and produce zero errors:

```
python - <<'PY'
import asyncio
from tests.unit.test_pentest_tools import FakeMsfrpcd, FakeNmap
from app.core.config import settings
from app.ai_engine.pentest import (
    ExecutionEngine, ExecutionScheduler, PentestContext,
    ScheduledModule, ScopeConfig, default_registry, default_tool_registry,
)

async def main():
    reg, tools = default_registry(), default_tool_registry()
    print("Attack Modules (8):", sorted(m.name for m in reg.get_all()))
    print("External Tools (2):", [t["name"] for t in tools.list_metadata()])

    msf_daemon = FakeMsfrpcd()
    msf_daemon.start()
    fake_nmap = FakeNmap()
    try:
        settings.MSF_ENABLED = True
        settings.MSF_RPC_HOST = "127.0.0.1"
        settings.MSF_RPC_PORT = msf_daemon.port
        settings.MSF_RPC_PASSWORD = "test-rpc-password"
        settings.MSF_RPC_SSL = False
        settings.NMAP_ENABLED = True

        scheduler = ExecutionScheduler(engine=ExecutionEngine(registry=reg))
        ctx = PentestContext(
            target="http://127.0.0.1:80/",
            scope=ScopeConfig(
                allowed_targets=["127.0.0.1"],
                allowed_ports=[22, 80, 443],
                max_impact="proof_of_concept",
            ),
            config={
                "nmap_runner": fake_nmap,
                "msf_module": "auxiliary/scanner/portscan/tcp",
                "msf_options": {"PORTS": "22,80,443"},
            },
        )

        plan = [
            ScheduledModule(module_name="nmap_scanner"),
            ScheduledModule(module_name="metasploit_scanner"),
        ]
        report = await scheduler.run(
            plan, ctx,
            on_progress=lambda pct, mod: print(f"  [PROGRESS {pct:>3}%] Finished {mod}"),
        )

        print(f"\nModules Run: {report.modules_run} | Errors: {report.errors}")
        print("Shared Nmap Ports in Context:", ctx.get_shared("nmap_open_ports"))
        for f in report.findings:
            print(f"  * [{f.severity.value.upper()}] ({f.module_name}) {f.title}")
            print(f"    open_ports={f.metadata.get('open_ports')} | services={f.metadata.get('services', 'N/A')}")
    finally:
        msf_daemon.stop()

asyncio.run(main())
PY
```

Expected output:

```
Attack Modules (8): ['credential_tester', 'metasploit_scanner', 'nmap_scanner', 'path_traversal_probe', 'port_scanner',
'service_fingerprint', 'sqli_probe', 'xss_probe']
External Tools (2): ['metasploit', 'nmap']
  [PROGRESS  50%] Finished nmap_scanner
  [PROGRESS 100%] Finished metasploit_scanner

Modules Run: 2 | Errors: []
Shared Nmap Ports in Context: [80, 443]
  * [INFO] (nmap_scanner) Nmap scan: 2 open ports on 127.0.0.1
    open_ports=[80, 443] | services=[{'port': 80, 'protocol': 'TCP', 'service': 'http', 'product': 'Apache httpd',
'version': '2.4.7', 'cpe': ['cpe:/a:apache:http_server:2.4.7']}, {'port': 443, 'protocol': 'TCP', 'service': 'https',
'product': 'nginx', 'version': '1.25.3', 'cpe': ['cpe:/a:f5:nginx:1.25.3']}]
  * [INFO] (metasploit_scanner) Metasploit auxiliary/scanner/portscan/tcp: 2 open ports on 127.0.0.1
    open_ports=[80, 443] | services=N/A
```

6 Run Real Nmap + Metasploit + XSS Probe Against Authorized Public Targets

Once `nmap` is installed on your MacBook (`brew install nmap`), run this in `backend/` to execute a **real live Nmap service scan** against `scanme.nmap.org` together with `metasploit_scanner` and `xss_probe` against `demo.testfire.net` :

```
python - <<'PY'
import asyncio
from tests.unit.test_pentest_tools import FakeMsfrpcd
from app.core.config import settings
from app.ai_engine.pentest import (
    ExecutionEngine, ExecutionScheduler, PentestContext,
    ScheduledModule, ScopeConfig, default_registry,
)

async def main():
    daemon = FakeMsfrpcd()
    daemon.start()
    try:
        settings.MSF_ENABLED = True
        settings.MSF_RPC_HOST = "127.0.0.1"
        settings.MSF_RPC_PORT = daemon.port
        settings.MSF_RPC_PASSWORD = "test-rpc-password"
        settings.MSF_RPC_SSL = False
        settings.NMAP_ENABLED = True
        settings.NMAP_PATH = "nmap"

        scheduler = ExecutionScheduler(engine=ExecutionEngine(registry=default_registry()))
        ctx = PentestContext(
            target="http://demo.testfire.net/search.jsp?query=test",
            scope=ScopeConfig(
                allowed_targets=["scanme.nmap.org", "demo.testfire.net", "127.0.0.1"],
                allowed_ports=[22, 80, 443],
                max_impact="proof_of_concept",
            ),
            config={
                "nmap_ports": [22, 80],
                "msf_module": "auxiliary/scanner/portscan/tcp",
                "msf_options": {"PORTS": "22,80,443"},
            },
        )
        plan = [
            ScheduledModule(module_name="nmap_scanner", targets=["http://scanme.nmap.org/"]),
            ScheduledModule(module_name="metasploit_scanner", targets=["http://127.0.0.1:80/"]),
            ScheduledModule(module_name="xss_probe", targets=["http://demo.testfire.net/search.jsp?query=test"]),
        ]
        report = await scheduler.run(plan, ctx)
        for i, f in enumerate(report.findings, 1):
            print(f"[{i}] [{f.severity.value.upper()}] ({f.module_name}) {f.title}")
            if f.module_name == "nmap_scanner":
                print("    Services:", f.metadata.get("services"))
    finally:
        daemon.stop()

asyncio.run(main())
PY
```

Verified live output:

```
[1] [INFO] (nmap_scanner) Nmap scan: 2 open ports on scanme.nmap.org
    Services: [{'port': 22, 'protocol': 'TCP', 'service': 'ssh', 'product': 'OpenSSH', 'version': '6.6.1p1 Ubuntu
2ubuntu2.13', 'cpe': ['cpe:/a:openbsd:openssh:6.6.1p1', 'cpe:/o:linux:linux_kernel']}, {'port': 80, 'protocol': 'TCP',
'service': 'http', 'product': 'Apache httpd', 'version': '2.4.7', 'cpe': ['cpe:/a:apache:http_server:2.4.7']}]
[2] [INFO] (metasploit_scanner) Metasploit auxiliary/scanner/portscan/tcp: 2 open ports on 127.0.0.1
[3] [HIGH] (xss_probe) Reflected XSS in parameter 'query'
```

7 Taking Full Advantage of `nmap_scanner` — Configuration & Shared State

Pass any of the options below inside `PentestContext.config` to customize Nmap scans per engagement:

`context.config` Key	Default	How to Use It & Safety Enforcement
`nmap_ports`	`scope.allowed_ports` or `[80, 443, 8080, 8443]`	Pass a list (`[22, 80, 443, 3306]`) or comma string (`"22,80,443"`). Every port is filtered through `ScopeConfig.is_port_allowed()` and capped at `PENTEST_MAX_PORTS` (50).
`nmap_service_version`	`True`	Adds `-sV --version-light` so Nmap probes open ports for service name, product, version, and CPE identifiers (e.g., `cpe:/a:apache:http_server:2.4.7`). Set `False` for fast port-only discovery.
`nmap_scripts`	`[]` (none)	Run safe NSE discovery scripts, e.g., `["http-title", "banner", "ssl-cert"]` . **Code-enforced gate:** `dos` scripts are refused unless `allow_destructive=True` ; `exploit/intrusive/brute/vuln/fuzzer` scripts are refused unless `max_impact="full"` .
`nmap_timing`	`4` (`-T4`)	Nmap timing template from `0` (paranoid) to `5` (insane); automatically clamped to `0..5` .
`nmap_timeout`	`120` seconds	Maximum seconds before the Nmap subprocess is terminated cleanly.

Cross-Tool Data Sharing (`context.shared_state`): After every scan, `nmap_scanner` writes `context.get_shared("nmap_open_ports")` (e.g. `[22, 80]`) and `context.get_shared("nmap_services")` (full service/version/CPE list) into the shared context so downstream modules and tools in the same engagement can inspect what Nmap discovered.

8 Safety Gates & Troubleshooting Reference

Symptom / Finding Title	Cause & Resolution
`INFO "Nmap integration disabled"`	`NMAP_ENABLED=false` (default). Set `NMAP_ENABLED=true` in `backend/.env` (or `settings.NMAP_ENABLED = True`).
`INFO "Nmap unavailable"`	The `nmap` binary is not on your `PATH` . Install via `brew install nmap` (macOS) or `winget install Insecure.Nmap` (Windows).
`ERROR "Intrusive Nmap script refused"`	Expected safety gate: scripts matching `exploit` , `intrusive` , `brute` , `vuln` , or `fuzzer` require `max_impact="full"` in `ScopeConfig` .
`ERROR "Destructive Nmap script refused"`	Expected safety gate: `dos` scripts are blocked whenever `allow_destructive=False` .
`OutOfScopeError` before scan starts	Expected safety gate: `SafetyGuard.preflight()` blocked a target that is not in `ScopeConfig.allowed_targets` .